IntelliSwap: An Adaptive Per-Page Compressor Selection Strategy for Simulated Memory Swap Compression

Goyal Eklavya Shyam Sunder
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
eklavyagoyal108@gmail.com

Aditya Agarwal
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
agg.aditya26@gmail.com

Sameer Kazmi
School of Computer Science and Engineering (SCOPE)
Vellore Institute of Technology
Vellore, India
sameerkazmi05@gmail.com

***Abstract*—The compressed swap back-ends in Linux, `zswap` and `zram`, run every evicted page through one compression algorithm chosen in advance by the administrator. Pages differ a great deal in how much redundancy they hold, so a single setting either spends CPU time on data that will not shrink or leaves memory unclaimed that a stronger compressor would have recovered. We present IntelliSwap, a userspace simulator that picks the compressor for each 4 KB page separately. For every page it computes three statistics in one pass (byte-level Shannon entropy, the fraction of zero bytes, and the mean run length), and a two-rule threshold selector then sends the page to LZ4 or to Zstandard at level 3. We tested the selector on 500 pages in four classes: zero-filled, random, text cut from real source and documentation files, and repetitive data made of raw image slices and constant-byte fills. The mean per-page compression ratio of the adaptive scheme is 79.07:1, against 79.08:1 for always-Zstd and 31.69:1 for always-LZ4, and its mean compression time drops from 7.72 μs to 5.65 μs per page, which is 26.8% less than always-Zstd. Counted over total bytes, the adaptive output is 1.8% larger than the always-Zstd output and 8.4% smaller than the always-LZ4 output. Most of the time saved comes from the 179 random and image pages that the selector keeps away from Zstd. A plain threshold rule, with no learned model, is therefore enough to keep most of Zstd's space advantage while avoiding much of its cost on pages that do not compress.**

***Index Terms*—memory compression, zswap, zram, adaptive compression, LZ4, Zstandard, operating systems, page classification, Shannon entropy**

I. Introduction

A. Background

When physical memory runs out, the kernel has two choices: kill a process, or move cold anonymous pages out to swap. Swapping to flash or an SSD keeps processes alive, but every later fault on a swapped page has to wait for block I/O, and interactive or latency-sensitive programs feel that delay directly. Linux offers two ways to cut down this storage traffic by keeping evicted pages in RAM in compressed form. `zswap` [1] is a compressed write-back cache that sits in front of an existing swap device, while `zram` [2] provides a RAM-backed block device that can be used as swap without needing any backing storage. In both, several logical pages share the space of one physical frame, and many swap-outs never reach the disk. Compressed swap is common on phones and has also been deployed across data-center fleets to reclaim cold memory [3].

Both subsystems support several compressors, including LZO, LZ4 [4] and Zstandard (Zstd) [5], but the setting applies to a whole pool or device. Once it is chosen, every page goes through the same algorithm, whatever its contents. Evicted pages are far from uniform, though. Freshly allocated heap regions are mostly zeros, buffers holding media or encrypted data look like random noise, and program data or text has a moderate amount of byte-level redundancy.

B. Problem Statement

A single compressor forces a trade-off between CPU time and memory that cannot be tuned away. LZ4 keeps swap-out latency low but leaves savings on the table for structured or mostly-zero pages. Zstd recovers more space on those pages, yet it also spends extra cycles on high-entropy pages that will not compress at all. The question we study is this: *can a 4 KB page be inspected cheaply before compression and sent to the compressor that suits it, so that the system keeps close to Zstd's ratio without paying Zstd's cost on pages it cannot shrink?*

C. Objectives

- Build a modular userspace pipeline that models the per-page compression step of `zswap` and `zram`.
- Find byte statistics that can be computed in one pass and that track how compressible a page is.
- Design a simple rule-based policy that maps those statistics to either LZ4 or Zstd.
- Measure the ratio and latency of the adaptive policy against static always-LZ4 and always-Zstd baselines.

D. Scope of the Project

IntelliSwap is a userspace proof of concept that tests per-page compressor selection on a controlled set of 500 pages, and it measures the compression step only. Integration with the

kernel's compressed-memory allocators (`zsmalloc`/`zbud`), swap-in decompression latency, and lock contention across cores are outside its scope; Section VII returns to them.

Section II reviews related work on compressed caching and adaptive swap. Section III describes the simulator and the feature definitions, and Section IV covers the implementation. Section V reports the results, a per-class breakdown, and how the routing rule evolved. Section VI concludes, and Section VII outlines future work.

II. Literature Survey / Related Work

In-memory page compression has a long history in both operating systems and computer architecture research. Table I lists the systems closest to our work, and the discussion below explains where IntelliSwap fits.

The two compressors in IntelliSwap sit at different points of the speed–ratio trade-off. LZ4 [4] is a byte-oriented LZ77 [12] variant designed for throughput. Zstandard [5] also uses LZ77-style match finding, but adds Huffman coding [13] and finite-state entropy coding, which gives clearly smaller output at a higher CPU cost.

Wilson *et al.* [6] showed early on that in-memory data has word-level regularities that a purpose-built compressor can exploit, and Tuduce and Gross [7] made the size of the compressed area adapt to demand. Later work moved the adaptivity to *which* pages are compressed and how. ezswap [9] skips pages it predicts will compress poorly, the far-memory system of Lagar-Cavilla *et al.* [3] targets cold pages, iSwap [10] takes reuse and application priority into account, and Ariadne [11] changes the compression granularity according to hotness. Outside the swap path, Krintz and Sucu [8] switched compression methods at run time for network streams, where transmission time hides the cost of deciding.

A. Research Gap Identified

Shannon's source coding theorem [14] ties achievable lossless compression to entropy, and ezswap [9] has already shown that page entropy is cheap enough to compute inside the swap path to decide admission. What none of the systems above does is use such a content signal to choose *between compressors* for each page. Swap subsystems either fix one compressor or use entropy only as a yes/no admission gate. Adaptive compression outside the kernel usually depends on file types, which anonymous memory does not have, or on learned models whose inference cost is hard to justify on a path where compressing a page takes a few microseconds. IntelliSwap addresses this gap by combining one-pass page statistics with a two-rule threshold selector that needs no model at all.

III. Proposed Methodology

A. System Overview

IntelliSwap is a three-phase pipeline that runs entirely in userspace. Phase 1 builds the 500-page corpus and computes features for each page. Phase 2 applies the routing policy, compresses every page with the adaptive choice and with both static baselines, and records sizes and times. Phase 3 turns the logs into per-class tables and figures. The phases pass data to each other only through CSV files, so the routing rule in Phase 2 can be changed and rerun on exactly the same pages.

B. Proposed Architecture

Fig. 1 shows the architecture, which has five components.

Page generator. Produces 500 pages of $n = 4096$ bytes, 125 in each of four classes chosen to cover the range of compressibility:

- *Zero:* pages of `0x00` bytes. So that the class is not 125 identical buffers, about 30% of these pages (35 in our run) have one byte at a random offset set to a random non-zero value.
- *Random:* pages filled with `os.urandom`, standing in for encrypted or already-compressed data.
- *Text:* consecutive 4 KB slices of reStructuredText files from the Linux kernel `Documentation` tree and Python source from the `requests` package. Fragments shorter than 200 bytes are skipped, and shorter final slices are padded with zeros.
- *Repetitive:* 62 slices of an uncompressed 8-bit grayscale BMP image (512×512 pixels) and 63 synthetic pages filled with a single randomly chosen byte value.

Feature extractor. For a page P of $n = 4096$ bytes, let c_b be the number of times byte value $b \in \{0, \ldots, 255\}$ occurs and $p_b = c_b/n$. One linear scan gives three features:

$$H(P) = -\sum_{b:\, c_b > 0} p_b \log_2 p_b \quad \text{(bits/byte)}, \tag{1}$$

$$z(P) = \frac{c_0}{n}, \qquad \bar{\ell}(P) = \frac{n}{k(P)}, \tag{2}$$

where $k(P)$ is the number of maximal runs of identical consecutive bytes. $H(P) \in [0, 8]$ is the first-order entropy, $z(P) \in [0, 1]$ is the zero-byte fraction, and $\bar{\ell}(P) \in [1, 4096]$ is the mean run length.

Adaptive selector. Applies two ordered threshold checks to $z(P)$ and $H(P)$, as defined in Section III-C2. Each decision costs two comparisons.

Compression engine and timing harness. Compresses each page with the selected compressor and with static LZ4 and Zstd (level 3). A 4 KB page compresses in a few microseconds, so a single timer reading is easily disturbed by scheduling noise; each configuration is therefore run seven times back to back and the median is kept.

Results logger and evaluator. Writes per-page results to `results.csv`, from which Phase 3 computes corpus-wide and per-class averages.

C. Design

1) Execution Flow: The benchmark runs in six steps: (1) generate the 500 page files and `manifest.csv`; (2) compute $H(P)$, $z(P)$ and $\bar{\ell}(P)$ for each page; (3) ask the selector for a compressor; (4) compress the page with the selected compressor, with LZ4 and with Zstd, seven times each; (5) write the compressed sizes and median times to `results.csv`; and (6) compute summary statistics and draw the plots.

TABLE I
SUMMARY OF RELATED WORK

Sl.	Paper / Source & Author	Year	Methodology	Drawbacks / Limitations
1	Linux `zswap` documentation [1]	2013	Compressed write-back cache in front of a swap device; one configured compressor for all pages.	Global compressor setting; no awareness of what a page contains.
2	Linux `zram` documentation [2]	2014	RAM-backed compressed block device used as swap; stores pages filled with one repeated value without compressing them.	One compressor per device; other incompressible pages still pass through the full compressor.
3	Wilson, Kaplan & Smaragdakis [6]	1999	Compressed caching of evicted pages using the WK family of compressors, built around word-level patterns in memory data.	A single fixed compressor; adapts the cache size, not the algorithm.
4	Tuduce & Gross [7]	2005	Linux compressed-memory system that resizes the compressed area at run time according to demand.	Adapts capacity only; the compressor does not change per page.
5	Krintz & Sucu [8]	2006	Chooses a compression method at run time from predicted compressibility and available bandwidth, for network data.	Built for network transfer, where transmission time hides the decision cost; not a microsecond-scale swap path.
6	Kim, Kim & Seo, "ezswap" [9]	2019	Extends `zswap` on Android; estimates compressibility from page entropy and caches only pages expected to compress well.	Entropy decides *whether* a page is compressed; admitted pages all use the same compressor.
7	Lagar-Cavilla *et al.* [3]	2019	Proactively compresses cold pages into `zswap` across warehouse-scale servers.	Focuses on finding cold pages and when to reclaim them; the compressor is fixed.
8	Wang, Liu & Xiao, "iSwap" [10]	2024	Learns page reuse patterns in the kernel; keeps latency-critical pages compressed in RAM and evicts low-priority pages.	Chooses between compression and eviction by application priority, not between compressors.
9	Liang *et al.*, "Ariadne" [11]	2025	Mobile `zram` scheme that varies compression chunk size with data hotness and decompresses ahead of use.	Adapts chunk size to access hotness rather than the algorithm to page content.

2) Selection Policy: LZ4 serves as the low-latency path and Zstd at level 3 as the high-ratio path. For a page P, the selector chooses compressor $M(P)$ as follows:

$$M(P) = \begin{cases} \text{Zstd}, & \text{if } z(P) > 0.9, \\ \text{LZ4}, & \text{if } z(P) \le 0.9 \text{ and } H(P) > 7.0, \\ \text{Zstd}, & \text{otherwise.} \end{cases} \quad (3)$$

The thresholds come from the feature distributions of the corpus. Every zero-class page has $z(P) \ge 0.9998$, and Zstd encodes an all-zero page in 19 bytes against 49 bytes for LZ4, so the first rule sends sparse pages to Zstd. Random pages lie between 7.95 and 7.96 bits/byte, while no text page goes above 5.60 and no repetitive page above 7.43. A page above 7.0 bits/byte has little redundancy left, so the second rule sends it to LZ4 and saves Zstd's search effort. Everything else goes to Zstd.

The extractor still records the mean run length $\bar{\ell}(P)$, but the final rule does not use it. An earlier version did, and Section V-D explains why that rule was dropped.

D. Technologies Used / System Requirements

1) Software Environment: The simulator is written in Python 3.10+ and uses the C-extension bindings `lz4.frame` and `zstandard`, Matplotlib for plots, and the standard `csv`, `math`, `time` and `collections` modules.

2) Hardware Environment: The experiments run on an ordinary x86-64 machine with no hardware accelerators. The whole 500-page benchmark finishes in under a second and needs about 5 MB of disk for the page files and CSV logs.

IV. IMPLEMENTATION

A. Description of Modules

The code is split into small scripts that follow the three phases. In Phase 1, `generate_pages.py` writes the 500 binary pages and `manifest.csv`, and `features.py` reads the manifest, computes $(H, z, \bar{\ell})$ and writes `features.csv`. Phase 2 is driven by `run_pipeline.py`, which takes the threshold rule from `decision_logic.py` (listed in Appendix A) and calls the compressors through `compress_engine.py`. That module wraps both libraries behind one function, `compress_page(data, method)`. Phase 3 consists of `aggregate_stats.py` and `generate_plots.py`, which read `results.csv` and produce the tables and figures.

B. Prototype Development

Two details mattered for accurate measurement. First, times below 10 μs are sensitive to interrupts and timer resolution, so each compression call is timed with `time.perf_counter()` over seven runs and the median is used. Second, a single `ZstdCompressor(level=3)`

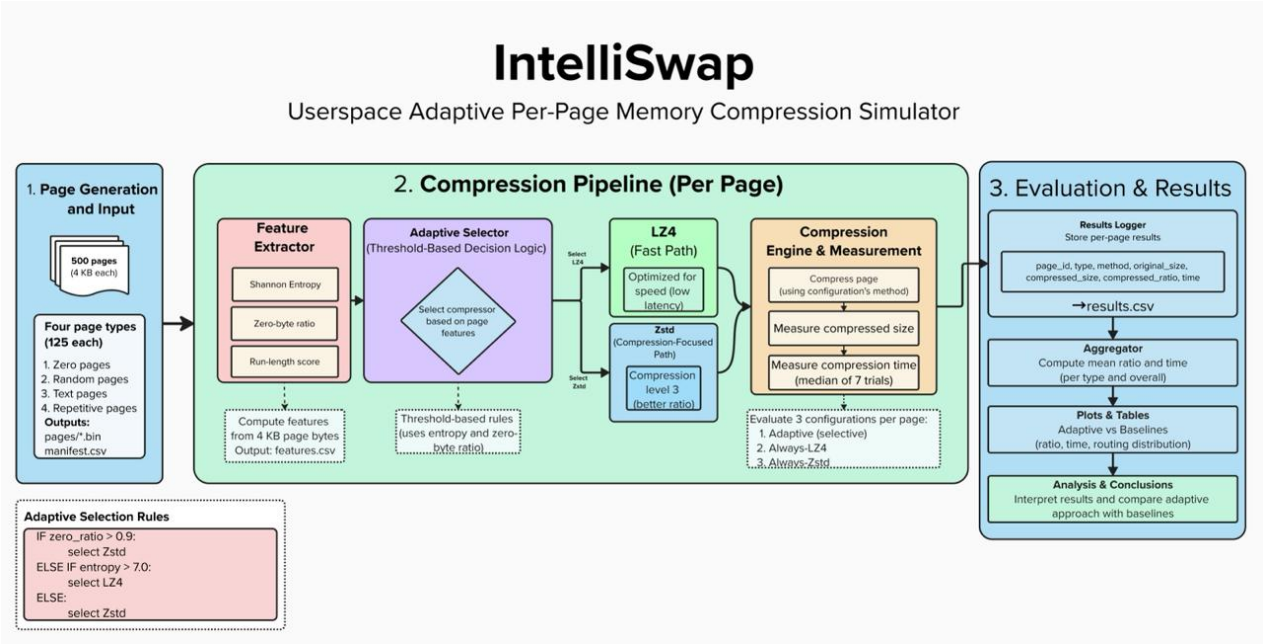


Fig. 1. IntelliSwap simulation pipeline. Phase 1 builds the 4 KB page corpus and computes single-pass byte features; Phase 2 routes each page to LZ4 or Zstd with threshold rules and benchmarks the static baselines; Phase 3 aggregates ratio and latency statistics.

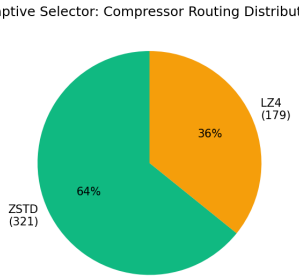


Fig. 2. Compressor selections made by the adaptive selector over the 500-page corpus.

object is created once at start-up and reused for every page, so that setup cost does not enter the measurements. Both libraries write complete frames, and every reported size includes the frame header.

C. Prototype Output

Fig. 2 shows how the corpus was routed: 321 pages (64.2%) went to Zstd and 179 (35.8%) to LZ4. All 125 zero pages and all 125 text pages went to Zstd, and all 125 random pages went to LZ4. The repetitive class was split. Its 63 constant-byte pages ($H = 0$) went to Zstd, and 54 of its 62 image slices, with entropies between 7.06 and 7.43 bits/byte, went to LZ4.

V. Experimental Results and Analysis

A. Experimental Setup

All runs used one machine and the same library versions on the 500-page corpus. The compression ratio of page P_i is $R_i = n/|C(P_i)|$, where $|C(P_i)|$ is the size of the compressed frame in bytes. Latency is the median of seven runs, in microseconds, and covers only the compressor call; the cost of feature extraction is discussed in Section V-F.

Per-page ratios range from about $0.99 : 1$ on random data to over $200 : 1$ on zero pages, so we report two summary ratios:

- **Mean per-page ratio** $\bar{R} = \frac{1}{N}\sum_{i=1}^{N} R_i$, the average over all $N = 500$ pages, plotted in Figs. 3 and 5.
- **Aggregate corpus ratio** $R_{\text{agg}} = \sum_i n / \sum_i |C(P_i)|$, total input bytes divided by total output bytes. $\bar{R}$ is pulled up strongly by the tiny frames of zero pages, so R_{agg} is the better guide to how much RAM a swap pool would actually save.

B. Results Obtained

Table II gives the corpus-wide results. IntelliSwap reaches $\bar{R} = 79.07 : 1$, practically the same as static Zstd ($79.08 : 1$) and $2.49\times$ the value for static LZ4 ($31.69 : 1$). Its mean compression time is 5.65 μs per page against 7.72 μs for always-Zstd, a 26.8% reduction.

In terms of total bytes, the 2,048,000-byte corpus compresses to 1,055,786 bytes with always-LZ4 ($R_{\text{agg}} = 1.94$), 949,937 bytes with always-Zstd (2.16), and 967,078 bytes with IntelliSwap (2.12). The adaptive output is therefore 1.8% larger than Zstd's and 8.4% smaller than LZ4's. Put another way, IntelliSwap keeps 83.8% of the bytes that Zstd saves over

TABLE II
OVERALL RESULTS ACROSS ALL 500 PAGES

Method	Mean $\bar{R}$	R_{agg}	Time (μs)	$\bar{R}$ vs LZ4
Adaptive (IntelliSwap)	79.07	2.12	5.65	2.49×
Always LZ4	31.69	1.94	2.32	1.00×
Always Zstd	79.08	2.16	7.72	2.50×

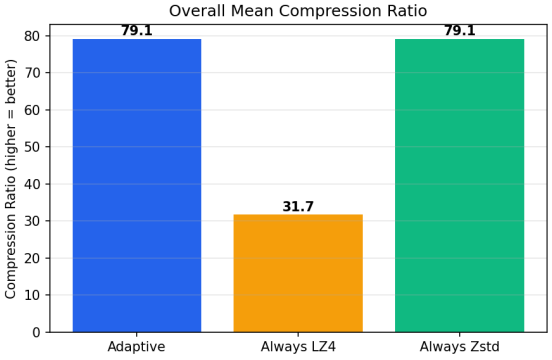


Fig. 3. Overall mean compression ratio ($\bar{R}$). IntelliSwap matches Always-Zstd and is well above Always-LZ4.

LZ4 while using about a quarter less compression time than Zstd.

C. Performance Analysis

Table III and Figs. 5 and 6 break the results down by page class.

Zero pages. All 125 satisfy $z(P) > 0.9$ and go to Zstd, giving $\bar{R} = 203.06 : 1$ against $80.95 : 1$ for LZ4. An all-zero page compresses to 19 bytes with Zstd ($215.6 : 1$) and 49 bytes with LZ4 ($83.6 : 1$); the class mean is a little lower because the 35 pages with one non-zero byte compress to about 24 bytes. In absolute terms Zstd saves only about 30 bytes per page here, mostly through its smaller frame. The gap between 1.79 μs and 1.74 μs is measurement variance, since both configurations run the same compressor.

Repetitive pages. This class shows the trade-off made by the $H > 7.0$ cutoff. The 63 constant-byte pages go to Zstd and behave like zero pages. The 54 image slices with H between 7.06 and 7.43 go to LZ4. Over all 62 image slices, Zstd reaches an aggregate ratio of only $1.08 : 1$, against $1.00 : 1$ for LZ4. Sending those 54 slices to LZ4 lowers the class mean ratio slightly, from 109.19 to 109.16, but cuts the class mean time from 8.29 μs to 2.94 μs, about 2.8 times faster.

Random pages. At close to 8 bits/byte, neither compressor can shrink these pages, and both end up slightly above the input size because of frame overhead ($R \approx 0.99$–1.00). Zstd still spends 5.38 μs looking for matches before storing the block raw, whereas LZ4 skips through unmatched input quickly and finishes in 1.51 μs. Routing these pages to LZ4 saves 3.87 μs each, and the LZ4 frame is only about 13 bytes larger than the Zstd frame.

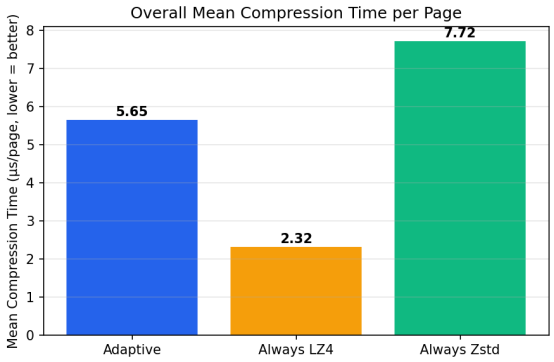


Fig. 4. Overall mean compression time per page. Sending high-entropy pages to LZ4 cuts the mean by 26.8% relative to Always-Zstd.

TABLE III
PER-CLASS MEAN RATIO AND TIME (μS)

Class	Ad. R	LZ4 R	Zstd R	Ad. μs	LZ4 μs	Zstd μs
Zero	203.06	80.95	203.06	1.79	1.12	1.74
Repetitive	109.16	42.62	109.19	2.94	1.54	8.29
Text	3.06	2.20	3.06	16.38	5.18	15.47
Random	0.99	0.99	1.00	1.51	1.46	5.38

Text pages. All 125 text slices have $H \leq 5.60$ and go to Zstd, which gives $\bar{R} = 3.06 : 1$ against $2.20 : 1$ for LZ4. They are also the most expensive pages, at roughly 15.5 to 16.4 μs under Zstd. Because the adaptive and always-Zstd configurations run the same compressor on the same bytes, the difference between 16.38 μs and 15.47 μs (about 5.9%) gives a rough noise floor for our timing harness.

Overall, the drop in mean time from 7.72 μs to 5.65 μs comes mostly from the 179 pages sent to LZ4. Each random page saves about 3.9 μs, and each of the 54 image slices saves roughly 12 μs by an estimate from the class means. Part of this gain is offset by timing noise on the text class.

D. Evolution of the Selection Rule

The final rule took three iterations, summarized in Table IV. The first version (R1) had an extra rule that sent pages with long byte runs to LZ4, and it also sent zero pages to LZ4, on the assumption that long runs are easy for any compressor. In practice this kept the overall mean ratio at about $31.9 : 1$, barely above always-LZ4, because Zstd encodes long runs far more compactly than LZ4. Removing the run-length rule (R2) let the constant-byte pages reach Zstd and raised $\bar{R}$ to $48.53 : 1$. Looking at the per-class results then showed that R2 still sent the 125 zero pages to LZ4 ($80.95 : 1$ against Zstd's $203.06 : 1$). Sending them to Zstd instead gave the final rule, R3, with $\bar{R} = 79.07 : 1$.

The step from R2 to R3 also shows why both summary ratios are needed. Moving the 125 zero pages to Zstd lifts $\bar{R}$ from 48.53 to 79.07, yet R_{agg} only moves from 2.11 to 2.12, because 30 bytes saved on each zero page add up to about 3.75 KB over a 2 MB corpus.

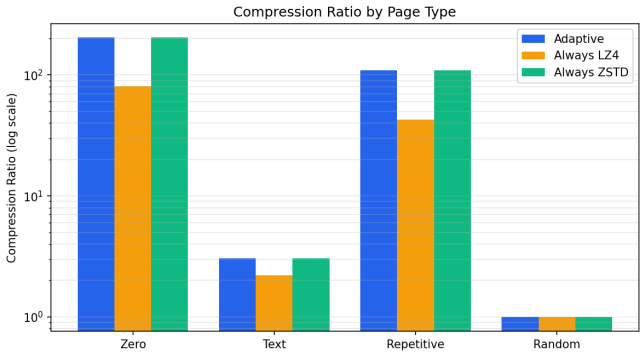


Fig. 5. Mean compression ratio by page class (log scale). IntelliSwap tracks Always-Zstd on every class and is above Always-LZ4 on every compressible class.

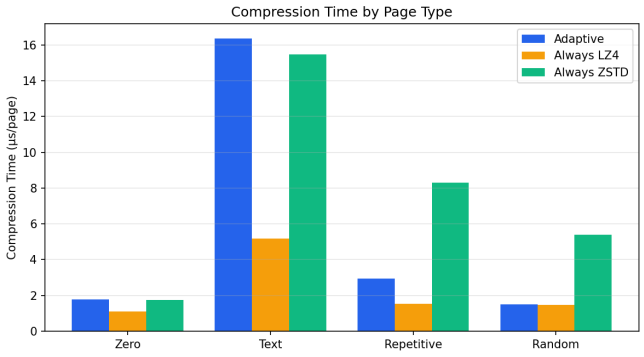


Fig. 6. Mean compression time by page class. Routing random and high-entropy image pages to LZ4 removes most of Zstd's cost on them.

E. Comparison with Static Baselines

Compared with a static `zswap` or `zram` setup using LZ4, IntelliSwap reduces the total compressed footprint by 8.4% and raises $\bar{R}$ by $2.49\times$, at an extra 3.33 μs per page. Compared with a static Zstd setup, it stays within 1.8% of Zstd's footprint and within 0.01% of its $\bar{R}$, while cutting the mean compression time by 26.8%.

F. Limitations and Threats to Validity

Four limitations apply when carrying these results over to a real kernel.

1) *Synthetic corpus and same-filled pages:* `zram` stores pages filled with one repeated value as metadata without running a compressor [2], and recent kernels also skip zero-filled pages before they reach `zswap`. On such a system most of our zero and constant-byte pages would never reach the selector. In addition, all image pages come from a single image, so the repetitive class is narrow.
2) *Feature-extraction overhead:* the compression timer excludes feature extraction, and our Python extractor says little about kernel cost. A C implementation of the histogram, entropy and zero-ratio computation, run on the same 500 pages on a 2.8 GHz Xeon virtual machine, took about 2.0 μs per page when scanning all 4096 bytes and about 0.64 μs when sampling every eighth byte. The full-page cost is close to the mean saving of 2.07 μs, so the net benefit depends on sampling. With a strided 512-byte sample and the entropy threshold lowered to 6.8 bits/byte to offset the downward bias of sampled entropy, routing matched the full-page decision on 499 of 500 pages. Because our compression times were measured through Python bindings, these figures are indicative rather than directly comparable.
3) *In-sample threshold:* the $H > 7.0$ cutoff was set after looking at the same 500 pages used for evaluation. It still has to be tested on held-out memory dumps from real applications.
4) *No decompression measurement:* swap-in faults depend on decompression speed, where LZ4 is clearly faster than Zstd. A complete policy should weigh how often a page is faulted back in, not only the cost of compressing it.

TABLE IV
ABLATION ACROSS SELECTION RULE ITERATIONS

Ver.	Rule set	LZ4/Zstd	$\bar{R}$	R_{agg}
R1	zero→LZ4; long runs→LZ4; $H>7$ →LZ4; else Zstd	–	≈31.9	–
R2	zero→LZ4; $H>7$ →LZ4; else Zstd	305/195	48.53	2.11
R3	zero→Zstd; $H>7$ →LZ4; else Zstd	179/321	79.07	2.12

VI. Conclusion

IntelliSwap is a userspace simulator that chooses between LZ4 and Zstandard for each page using one-pass byte statistics. On 500 pages of zero-filled, random, text and repetitive data, a two-rule selector on zero-byte fraction and entropy matched the mean ratio of static Zstd (79.07 : 1 against 79.08 : 1), stayed within 1.8% of Zstd's total compressed size, and reduced mean compression time by 26.8%. Against static LZ4 it produced 8.4% less compressed data. Almost all of the CPU saving came from keeping high-entropy pages away from Zstd. Within the limits listed above, the results suggest that inspecting a page before compressing it is a practical way to avoid wasted work in compressed swap, provided the inspection itself is kept cheap, for example by sampling.

VII. Future Scope

Several directions follow from this work: (1) implement the histogram-based selector in C inside the `zswap` compression path and measure the net end-to-end CPU cost, together with its interaction with `zsmalloc` and with lock contention across cores; (2) study sampled feature extraction further, since a 512-byte strided sample already reduced the cost to about 0.64 μs per page in our C measurement; (3) add an explicit same-filled-page bypass and further compressors such as LZO-RLE and Zstd level 1; (4) include swap-in decompression

latency in the routing decision for frequently faulted pages; and (5) validate the thresholds on memory dumps from virtual machines and browser heaps.

APPENDIX A
CORE DECISION LOGIC

```
# decision_logic.py -- adaptive per-page selection
def choose_compressor(entropy, zero_ratio,
                      run_length_score):
    # zero-heavy page: smaller Zstd frame
    if zero_ratio > 0.9:
        return "zstd"
    # high entropy: skip Zstd's extra work
    if entropy > 7.0:
        return "lz4"
    # text / patterned data: use Zstd
    return "zstd"
```

REFERENCES

[1] The Linux Kernel Documentation, "zswap," Linux Kernel Organization. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/mm/zswap.html

[2] The Linux Kernel Documentation, "zram: Compressed RAM-based block devices," Linux Kernel Organization. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/blockdev/zram.html

[3] A. Lagar-Cavilla *et al.*, "Software-defined far memory in warehouse-scale computers," in *Proc. 24th Int. Conf. Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2019, pp. 317–330, doi: 10.1145/3297858.3304053.

[4] Y. Collet, "LZ4 – Extremely fast compression algorithm," 2011. [Online]. Available: https://lz4.github.io/lz4/

[5] Y. Collet and M. Kucherawy, Eds., "Zstandard compression and the 'application/zstd' media type," RFC 8878, IETF, Feb. 2021.

[6] P. R. Wilson, S. F. Kaplan, and Y. Smaragdakis, "The case for compressed caching in virtual memory systems," in *Proc. USENIX Annual Technical Conf.*, 1999, pp. 101–116.

[7] I. C. Tuduce and T. Gross, "Adaptive main memory compression," in *Proc. USENIX Annual Technical Conf.*, 2005, pp. 237–250.

[8] C. Krintz and S. Sucu, "Adaptive on-the-fly compression," *IEEE Trans. Parallel Distrib. Syst.*, vol. 17, no. 1, pp. 15–24, Jan. 2006.

[9] J. Kim, C. Kim, and E. Seo, "Ezswap: Enhanced compressed swap scheme for mobile devices," *IEEE Access*, vol. 7, pp. 139678–139691, 2019, doi: 10.1109/ACCESS.2019.2942362.

[10] Z. Wang, L. Liu, and L. Xiao, "iSwap: A new memory page swap mechanism for reducing ineffective I/O operations in cloud environments," *ACM Trans. Archit. Code Optim.*, vol. 21, no. 3, Art. no. 47, 2024, doi: 10.1145/3653302.

[11] Y. Liang *et al.*, "Ariadne: A hotness-aware and size-adaptive compressed swap technique for fast application relaunch and reduced CPU usage on mobile devices," in *Proc. IEEE Int. Symp. High-Performance Computer Architecture (HPCA)*, 2025.

[12] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. Inf. Theory*, vol. 23, no. 3, pp. 337–343, May 1977.

[13] D. A. Huffman, "A method for the construction of minimum-redundancy codes," *Proc. IRE*, vol. 40, no. 9, pp. 1098–1101, Sep. 1952.

[14] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, 1948.